

## 〈C프로그래밍 및 실습〉 2장 실습

### [ 문제 5 ]

- 두 개의 정수를 저장할 변수 N과 M을 선언하고, scanf()문을 이용해 두 정수를 입력받아 변수 N과 M에 저장합니다.
- 변수 max와 min을 선언하고, 변수 N과 M에 저장된 정수 중 큰 수를 변수 max로, 작은 수를 변수 min으로 복사합니다.
- 큰 수와 작은 수를 계산하기 위해 조건 연산자를 다음과 같이 사용합니다.

```
max = (  ) ? N : M;  
// 네모 안의 수식이 참이면, max=N이 되고, 거짓이면, max=M이 되도록 네모 안의 수식을 완성하세요.
```

▶ 같은 방식으로 두 수 중 작은 수를 min에 저장하는 수식을 조건 연산자를 사용하여 작성합니다.

- max를 min으로 나눈 몫과 나머지를 계산해서 출력합니다.

## 〈C프로그래밍 및 실습〉 3장 실습

### [ 문제 1 ]

- 입력받은 정수를 저장할 변수 num과 입력된 5개 정수 중 양수들만의 합을 저장할 변수 sum을 선언합니다.

```
int num, sum = 0; // 변수 sum에는 입력된 정수 중 양수의 합이 누적되어 계산되므로, 0으로 초기화.
```

- scanf()문으로 정수를 하나 입력받아, 변수 num에 저장하고, 변수 num에 저장된 값이 양수이면, 변수 num의 값을 변수 sum에 더합니다.

```
scanf("%d", &num);  
if ( num이 양수이면 ) { // num에 저장된 정수가 양수인지 확인하는 조건 만들기  
    num의 값을 sum에 더하기;  
}
```

▶ 같은 작업을 5번 실행해서, 5개 정수를 입력받고, 그 중 양수만 sum에 더해 출력합니다.

## [ 문제 2 ]

- 변수 ch를 선언하고, 문자 하나를 scanf()문으로 입력받아 변수 ch에 저장합니다.

```
if ( ch에 저장된 문자가 대문자이면 ) {    // ch가 소문자인 조건 만들기 (3주차 2장 실습 [ 문제 4 ] 참고)
    대문자를 소문자로 바꾸어 출력;        // 2주차 1장 실습#2 [ 문제 6 ] 참고
}
```

▶ 같은 방식으로 ch에 저장된 문자가 소문자이면, 대문자로 바꾸어 출력합니다.

- 변수 ch에 저장된 문자가 대문자도 아니고, 소문자도 아니면, "none"을 출력 합니다.

```
if ( ch에 저장된 문자가 대문자도 아니고 소문자도 아니면 ) {
    // ch가 대문자도 아니고, 소문자도 아닌 조건 만들기 ( !대문자인 조건 ) && !(소문자인 조건 )
    printf("none\n");
}
```

## [ 문제 3 ]

- 문자 두 개를 입력받아 저장할 변수 ch1과 ch2를 선언하고, scanf()문으로 문자 2개를 입력받아, ch1과 ch2에 저장합니다.

```
scanf("%c-%c", &u, &l);    // 입력 형식이 "대문자-소문자"라 "%c"와 "%c" 사이에 "-" 표시 필요
```

- 알파벳 순서상 먼저 나오는 문자란, 크기가 작은 문자를 의미합니다.

- ▶ 먼저, ch1에 저장된 대문자의 순서가, ch2에 저장된 소문자의 순서보다 작거나 같으면, ch1에 저장된 소문자를 출력합니다.
- ▶ 변수에 저장된 대문자 또는 소문자가 몇 번째 문자인지 계산하는 법은 "2주차 1장 실습 #2 [ 문제 3 ] 참고"

```
if ((ch1이 몇 번째 대문자인지 계산) <= (ch2가 몇 번째 소문자인지 계산)){
    // ch1에 저장된 대문자의 순서가, ch2에 저장된 소문자의 순서보다 작거나 같으면
    printf("%c\n", ch1);    // ch1에 저장된 대문자 출력
}
```

- ▶ 같은 방식으로 ch1에 저장된 대문자의 순서가, ch2에 저장된 소문자의 순서보다 크면, ch2에 저장된 소문자를 출력합니다.

## [ 문제 4 ]

- 정수 4개를 입력받아 저장할 변수 n1, n2, n3, n4를 선언하고, scanf()문으로 정수 4개를 입력받아, n1, n2, n3과 n4에 저장합니다.
- 첫 번째 정수보다 두 번째 정수가 크면, "UP"을, 작으면, "DOWN"을 출력합니다. 같으면, 아무것도 출력하지 않습니다.

```

if (n1이 n2 보다 작으면) {
    printf("UP ");
    flag = 1;
}
if (n1이 n2 보다 크면) {
    printf("DOWN ");
    flag = 1;
}

```

▶ 같은 방식으로 n2와 n3, 그리고, n3와 n4의 크기를 비교하여, "UP" 또는 "DOWN"을 출력합니다.

- 입력된 4개의 정수가 모두 같아, "UP" 또는 "DOWN"의 출력이 한 번도 없으면, "NONE"을 출력합니다.

- ▶ 간단한 방법은 4개 정수의 값이 같은지 "==" 연산자로 확인을 해보는 겁니다.
- ▶ 그러나, 이 문제에서는 "flag"를 사용하는 다른 방법을 배워 봅시다.

```

int flag = 0;    // flag 변수를 선언하고, 0으로 초기화

// 조건에 따른 "UP" 또는 "DOWN" 출력

if (flag==0){    // 출력을 마친 후, flag가 여전히 0이면, 실제 출력은 없었음
                // 출력이 하나라도 있으면, 위 박스의 파란색으로 표시한 flag=1;이 실행
                // "NONE" 출력
}

```

## [ 문제 6 ]

- 정답을 나타내는 3개의 정수를 저장할 변수 x1, x2, x3와 추측한 값을 저장할 3개의 변수 y1, y2, y3를 선언하고, 두 개의 scanf()문으로 정답을 나타내는 3개 정수와 추측한 값을 나타내는 3개의 정수를 입력받아 x1, x2, x3와 y1, y2, y3에 저장합니다.
- 스트라이크와 볼을 카운트할 2개의 변수 s와 b를 선언하고, 두 변수는 0으로 초기화합니다. **(덧셈 결과를 저장할 변수 또는 특정한 상황을 세기 위한 카운터로 사용하는 변수는 반드시 선언과 동시에 0으로 초기화 합니다.)**
- 정답의 첫 번째 정수부터 하나씩 다음 상황을 확인하고, 스트라이크 또는 볼의 개수를 셉니다.

```

if (x1 == y1) s++;    // 값과 위치가 일치, 스트라이크
if (x1 == y2) b++;    // 값만 일치, 볼
if (x1 == y3) b++;    // 값만 일치, 볼

```

- ▶ 같은 방식으로 x2와 x3 값에 대한 스트라이크와 볼의 개수를 세어 출력합니다.